Development Standards
& Design Guide

React · TypeScript · Tailwind CSS

v1.0 — March 2026

Rules where it matters. Flexibility everywhere else.

1. Project Structure

This is the canonical folder layout. Deviate only with a stated reason in a PR comment.

```
src/
  components/        # Reusable UI components
    ui/              # Primitives (Button, Input, Card)
    layout/          # Layout wrappers (Shell, Sidebar, Nav)
    [feature]/       # Feature-scoped components
  hooks/             # Custom React hooks
  lib/               # Utility functions, API clients
  types/             # Shared TypeScript types & interfaces
  pages/             # Route-level components (or app/ if Next.js)
  styles/            # Global styles, Tailwind config extensions
  constants/         # App-wide constants & enums
  context/           # React context providers
  assets/            # Static assets (images, icons)
  __tests__/         # Test files (mirror src/ structure)
```

Rules

- **One component per file.** The filename matches the default export. PascalCase for components, camelCase for hooks and utils.
- **Co-locate tests.** Place test files next to the source file or in a mirrored __tests__/ folder. Either convention is fine — pick one and stick with it.
- **No barrel exports (index.ts re-exports) in feature folders.** They slow down bundlers and create circular dependency headaches. Import directly from the file.
- **Shared types go in types/.** Types used by only one component live in that component's file.

2. TypeScript Conventions

Strict Mode

Enable `strict: true` in tsconfig.json. No exceptions. This gives you noImplicitAny, strictNullChecks, and the rest of the strict family.

Types vs Interfaces

Use	When
interface	Object shapes, component props, API responses — anything that benefits from declaration merging or extension
type	Unions, intersections, mapped types, utility types, or anything that isn't a plain object shape

Don't overthink it. If you're defining props, use interface. For everything else, type is fine.

Naming

Entity	Convention	Example
Component	PascalCase	UserCard.tsx
Hook	camelCase, use* prefix	useAuth.ts
Utility function	camelCase	formatDate.ts
Type / Interface	PascalCase	UserProfile
Enum	PascalCase, singular	Status (not Statuses)
Constant	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
Boolean prop/var	is/has/should prefix	isLoading, hasError

Avoid

- `any` — use `unknown` if you truly don't know the type, then narrow it.
- `as` type assertions — prefer type guards. `as` silences the compiler when you should be proving the type.
- Non-null assertions (`!`) — handle the null case explicitly.
- `enum` for string unions — use `type Status = "active" | "inactive"` unless you need reverse mapping.

3. React Patterns

Component Structure

Every component file follows this order:

```
// 1. Imports
// 2. Types / interfaces
// 3. Constants (component-scoped)
// 4. Component definition (export default function)
// 5. Sub-components (if small and tightly coupled)
// 6. Helper functions (component-scoped)
```

Functional Components Only

No class components. No reason to use them in 2026. Arrow functions or function declarations are both fine — be consistent within a file.

Props

- Destructure props in the function signature.
- Use `interface` for props. Name it `[ComponentName]Props`.
- Default values in destructuring, not `defaultProps`.

```
interface UserCardProps {
  name: string;
  role?: string;
  isActive?: boolean;
}

export default function UserCard({
  name,
  role = "member",
  isActive = true
}: UserCardProps) {
  // ...
}
```

State Management

Scope	Tool
Component-local	useState, useReducer
Shared across a subtree	React Context + useContext
Global / complex	Zustand (preferred) or Redux Toolkit. Pick one and commit.
Server state	TanStack Query (React Query). Never roll your own caching.

Don’t reach for global state until you’ve confirmed local state and prop drilling won’t work. Most features don’t need global state.

Custom Hooks

- Extract logic into a hook when two or more components share it.
- Hooks must start with `use`. No exceptions — React's rules of hooks depend on it.
- Keep hooks focused. `useAuth` should not also manage toast notifications.

4. Tailwind CSS & Styling

Core Principles

- **Utility-first.** Write Tailwind classes directly on elements. Don't create .css files for component styles unless you're building a genuinely reusable primitive with 10+ variants.
- **Design tokens via tailwind.config.** All colors, spacing, font sizes, and breakpoints are defined in the Tailwind config. Never hardcode hex values or pixel sizes in JSX.
- **Consistent ordering.** Install and use the Prettier Tailwind plugin (`prettier-plugin-tailwindcss`) to auto-sort class names. Eliminates arguments about ordering.

Color System

Define a semantic color palette in your Tailwind config. Map intent to color, not color to component.

Token	Usage	Example Tailwind Class
primary	CTAs, links, active states	bg-primary, text-primary
secondary	Supporting actions, badges	bg-secondary
destructive	Delete, error states	bg-destructive, text-destructive
muted	Disabled, placeholder, borders	bg-muted, text-muted-foreground
background	Page/card backgrounds	bg-background
foreground	Primary text	text-foreground

This means you change your brand color in one place (tailwind.config.ts) and it propagates everywhere.

Responsive Design

- **Mobile-first.** Default styles target mobile. Add breakpoint prefixes (`sm:`, `md:`, `lg:`) to scale up.
- **Standard breakpoints:** `sm: 640px`, `md: 768px`, `lg: 1024px`, `xl: 1280px`. Don't add custom breakpoints unless you have a real edge case.
- **Test at breakpoints, not just on your monitor.** Use browser devtools responsive mode.

What Not To Do

- Don't use `@apply` to build component classes. It defeats the purpose of utility-first and creates hidden coupling.
- Don't use inline `style={{}}` unless you're computing a dynamic value (like a width from data). Static styles belong in Tailwind classes.

- Don’t use `!important` via `!` prefix. If you need it, your specificity is wrong.

Dark Mode

If the project supports dark mode, use Tailwind’s `dark:` variant with class-based toggling (`darkMode: 'class'` in config). Every color token should have a dark counterpart defined in your CSS variables.

5. Visual Design System

Typography

Element	Size	Weight	Tailwind Class
Page title	30px / 1.875rem	Bold (700)	text-3xl font-bold
Section heading	24px / 1.5rem	Semibold (600)	text-2xl font-semibold
Subsection	20px / 1.25rem	Semibold (600)	text-xl font-semibold
Body	16px / 1rem	Normal (400)	text-base
Small / caption	14px / 0.875rem	Normal (400)	text-sm
Code	14px / 0.875rem	Normal (400)	text-sm font-mono

Line height: use Tailwind's `leading-normal` (1.5) for body text and `leading-tight` (1.25) for headings.

Spacing

Stick to Tailwind's spacing scale. The 4px base unit keeps everything aligned.

Use Case	Scale	Pixels
Tight inline spacing	1–2	4–8px
Between related elements	2–3	8–12px
Between sections/cards	4–6	16–24px
Page-level padding	6–8	24–32px
Major section gaps	12–16	48–64px

Border Radius

Element	Radius	Class
Buttons, inputs, badges	6px	rounded-md
Cards, modals, panels	8–12px	rounded-lg or rounded-xl
Avatars, status dots	Full circle	rounded-full

Pick one card radius and use it everywhere. Visual consistency comes from repetition.

Shadows & Elevation

- `shadow-sm` — subtle lift for cards and inputs

- `shadow-md` — dropdowns, popovers
- `shadow-lg` — modals, dialogs

Don’t stack shadows. One elevation level per element.

6. Component Standards

UI Primitives

Build or adopt these foundational components first. Everything else composes from them.

Component	Variants / States	Notes
Button	primary, secondary, destructive, ghost, outline + disabled, loading	Always use <button> for actions, <a> for navigation
Input	text, email, password, search + error, disabled	Wrap with label. Always include aria-describedby for errors
Card	default, outlined, elevated	Consistent padding (p-4 or p-6). No inline styles
Badge	default, success, warning, error, neutral	Use for status indicators. Keep text short (1–3 words)
Modal / Dialog	default + sizes (sm, md, lg)	Trap focus. Close on Escape. Use Radix or Headless UI
Toast	success, error, info, warning	Auto-dismiss with configurable duration. Stack from bottom-right

Accessibility (Non-Negotiable)

These aren't suggestions. Ship accessible or don't ship.

- All interactive elements must be keyboard-navigable (Tab, Enter, Escape).
- All images have `alt` text. Decorative images use `alt=""`.
- Form inputs have associated `<label>` elements. Placeholder text is not a label.
- Color contrast meets WCAG AA (4.5:1 for normal text, 3:1 for large text).
- Use semantic HTML: `<nav>`, `<main>`, `<section>`, `<article>`. Don't `<div>` everything.
- For complex interactive components (dropdowns, tabs, accordions), use Radix UI or Headless UI. Don't build your own ARIA patterns from scratch.

7. Code Quality & Tooling

Linting & Formatting

Tool	Config	Purpose
ESLint	eslint-config-next or @typescript-eslint/recommended	Catch bugs, enforce patterns
Prettier	Default config + tailwind plugin	Consistent formatting, auto-sorted Tailwind classes
TypeScript	strict: true	Type safety

Run `eslint` and `prettier` as pre-commit hooks via Husky + lint-staged. No broken code enters the repo.

Git Conventions

Branch Naming

```
feature/user-profile
fix/login-redirect
chore/update-deps
refactor/auth-flow
```

Commit Messages

Use conventional commits. Keep the subject line under 72 characters.

```
feat: add user avatar upload
fix: prevent duplicate form submission
refactor: extract auth logic into useAuth hook
chore: upgrade tailwindcss to 4.x
```

Pull Requests

- Keep PRs small. If it's touching 15+ files, break it up.
- Include a one-line summary of what and why.
- At least one review before merge. Self-merge is fine for typo-level fixes.

Testing

Layer	Tool	What to Test
Unit	Vitest	Utility functions, hooks, pure logic
Component	React Testing Library	User-facing behavior (not implementation details)
E2E	Playwright (if needed)	Critical user flows (auth, checkout, etc.)

Don’t chase coverage numbers. Test behavior that matters: user interactions, data transformations, edge cases. If a test would break from a CSS class rename, it’s testing the wrong thing.

8. Performance Guidelines

These aren't premature optimizations — they're habits that prevent problems.

- **Lazy-load routes.** Use `React.lazy()` + `Suspense` for route-level code splitting. No user should download the settings page on first load.
- **Memoize expensive computations.** Use `useMemo` for derived data that's costly to compute. Don't memoize everything — React's reconciliation is fast enough for most cases.
- **Avoid re-renders from unstable references.** Don't create objects or arrays inline in JSX if they're passed as props to memoized children. Hoist them or wrap with `useMemo`.
- **Images: use next/image or explicit width/height.** Prevents layout shift (CLS). Use WebP or AVIF formats where possible.
- **Bundle analysis.** Run `next build --analyze` (or equivalent) periodically. Catch unexpectedly large dependencies before they ship.

9. API & Data Patterns

API Calls

- Centralize API calls in `lib/api/`. Components should never contain raw `fetch()` calls.
- Use TanStack Query for server state. It handles caching, deduplication, background refetching, and error/loading states. Don't build this yourself.
- Type all API responses. Create interfaces that match the backend contract.

Error Handling

- Every API call has error handling. No uncaught promises.
- Use error boundaries at the route level to catch rendering failures gracefully.
- User-facing errors should be human-readable. Never show raw error objects or stack traces.

Environment Variables

- Prefix client-exposed vars with `NEXT_PUBLIC_` (Next.js) or `VITE_` (Vite).
- Never commit `.env` files. Use `.env.example` as a template with placeholder values.
- Validate env vars at startup (use zod or a simple check). Don't let the app silently run with missing config.

10. Decision Log

Track significant technical decisions here. When someone asks “why do we do it this way?” — point them here.

Decision	Choice	Rationale
State management	Zustand	Minimal boilerplate, TypeScript-native, no providers needed
Server state	TanStack Query	Handles caching, dedup, background refetch out of the box
Component library base	Radix UI + Tailwind	Accessible primitives, full styling control, no CSS-in-JS overhead
CSS approach	Tailwind utility-first	Fast iteration, consistent design tokens, great DX with IntelliSense
Testing	Vitest + RTL	Fast, ESM-native, same API as Jest but better TS support
Formatting	Prettier (default)	Zero config debates. Just format on save.
(Add your own)		

This is a living document. Update it when conventions change. If a rule isn’t working, change the rule — don’t silently ignore it.